Algorithmique Fondamentale : Recherche Séquentielle et Dichotomique dans les Tableaux

Thème principal : Algorithmes de recherche dans les structures linéaires et analyse de complexité temporelle

Synthèse exécutive : Ce cours aborde les méthodes fondamentales pour retrouver une valeur au sein d'une structure de données linéaire (tableau). L'enjeu pédagogique central réside dans la compréhension de l'impact direct de l'organisation des données (tableau trié ou non trié) sur la performance de l'algorithme. La recherche séquentielle effectue un parcours linéaire exhaustif en O(n), universel mais lent sur de grands volumes. À l'inverse, la recherche dichotomique tire parti d'un tableau trié pour appliquer le paradigme 'diviser pour régner', réduisant drastiquement le nombre de comparaisons à une échelle logarithmique O(log n). L'étudiant doit maîtriser les préconditions de chaque méthode, le calcul rigoureux des indices et l'analyse de complexité au pire des cas.

1. La Recherche Séquentielle (ou Linéaire)

La recherche séquentielle est l'approche la plus intuitive et universelle pour localiser un élément dans un tableau. Elle consiste à passer en revue chaque case du tableau, du premier indice (0) jusqu'au dernier (n-1), en comparant à chaque étape la valeur contenue dans la case avec la valeur recherchée (cible). L'algorithme s'arrête immédiatement en cas de succès (élément trouvé) ou lorsqu'il a parcouru la totalité du tableau sans succès. Si le tableau contient n éléments, le pire des cas se produit lorsque la valeur se trouve à la toute dernière position ou est totalement absente du tableau : l'algorithme doit alors effectuer exactement n comparaisons. La complexité est donc linéaire, notée O(n). Son principal avantage est qu'elle ne nécessite aucun pré-traitement ni organisation préalable des données.

- Parcours exhaustif élément par élément de l'indice 0 jusqu'à n-1
- Complexité temporelle linéaire O(n) au pire des cas
- Absence totale de précondition : fonctionne sur tableau trié ou non trié

Cas concret enseignant : *Métaphore du professeur : Recherche d'une clé tombée dans un tas de feuilles mortes. N'ayant aucun ordre ni rangement dans le tas, il est obligatoire de soulever chaque feuille une à une, de la première à la dernière, jusqu'à trouver la clé.*

2. La Recherche Dichotomique (Diviser pour Régner)

La recherche dichotomique applique le principe algorithmique 'diviser pour régner'. Elle exige une condition préalable stricte : le tableau doit être trié au préalable. Au lieu d'examiner les éléments un par un, l'algorithme calcule l'indice du milieu de la zone de recherche courante : milieu = (debut + fin) // 2. Il compare ensuite la valeur cible avec la valeur située à cet indice milieu. Trois cas se présentent : 1) Si T[milieu] == cible, l'élément est trouvé. 2) Si la cible est plus petite que T[milieu], la valeur ne peut se trouver que dans la moitié gauche : on restreint la recherche en déplaçant le pointeur fin à 'milieu - 1'. 3) Si la cible est plus grande, elle ne peut se trouver que dans la moitié droite : on déplace le pointeur debut à 'milieu + 1'. En éliminant la moitié des candidats restants à chaque étape, l'espace de recherche diminue de façon exponentielle, garantissant une complexité logarithmique O(log n).

- Précondition absolue : Le tableau doit être strictement trié au préalable
- Complexité temporelle logarithmique O(log2 n) au pire des cas
- Mise à jour dynamique de 3 pointeurs d'indices : debut, fin, milieu = (debut + fin) // 2

Cas concret enseignant : *Exemple du professeur : Le jeu du 'Devine un nombre entre 1 et 100'. Interroger 1, 2, 3... est une stratégie séquentielle inefficace. Proposer 50 permet d'éliminer d'un coup 50 valeurs si la réponse est 'c'est plus grand'. On trouve la réponse en 7 étapes maximum (log2(100) ≈ 6,64).*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne pas toujours utiliser la recherche dichotomique si elle est beaucoup plus rapide que la recherche séquentielle ?
Réponse du professeur : La recherche dichotomique nécessite impérativement un tableau trié. Si le tableau d'origine n'est pas trié, le trier coûte au minimum O(n log n) opérations. Pour effectuer une seule recherche ponctuelle dans un tableau non trié, la recherche séquentielle en O(n) est donc plus rapide et plus économe en calculs que 'trier puis chercher'.

Question étudiant : Comment calcule-t-on l'indice du milieu si le sous-tableau contient un nombre pair d'éléments ?
Réponse du professeur : On utilise la division entière (notée // en Python). La formule 'milieu = (debut + fin) // 2' arrondit automatiquement le résultat à l'entier inférieur, garantissant ainsi un indice entier valide dans le tableau.

■ Exercice & Démonstration d'Application

Énoncé : Soit le tableau trié T = [3, 8, 12, 15, 19, 24, 31, 42, 50, 61] comportant n = 10 éléments (indices 0 à 9). Appliquer pas à pas l'algorithme de recherche dichotomique pour rechercher la valeur 42.

Résolution pas à pas :
Étape 0 (Initialisation) : debut = 0, fin = 9. Étape 1 : Calcul du milieu -> milieu = (0 + 9) // 2 = 4. Valeur T[4] = 19. Comparaison : 42 > 19. La valeur recherchée est strictement supérieure, elle se trouve donc dans la partie droite. Mise à jour de début : debut = milieu + 1 = 5. Étape 2 : Nouvelle zone : debut = 5, fin = 9. Calcul du milieu -> milieu = (5 + 9) // 2 = 7. Valeur T[7] = 42. Comparaison : T[7] == 42. L'élément recherché est trouvé !

Résultat : La valeur 42 a été localisée à l'indice 7 en seulement 2 comparaisons, alors qu'une recherche séquentielle aurait demandé 8 comparaisons.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la condition impérative indispensable pour pouvoir exécuter une recherche dichotomique ?

R : Le tableau doit être préalablement trié de manière stricte (croissante ou décroissante).

Q : Quelle est la complexité temporelle au pire des cas d'une recherche séquentielle sur un tableau de taille n ?

R : La complexité est linéaire, notée O(n).

Q : Combien de comparaisons au maximum la recherche dichotomique nécessite-t-elle pour localiser une valeur dans un tableau trié de 1024 éléments ?

R : Il faut au maximum 10 comparaisons, car log2(1024) = 10 (à chaque étape, l'espace de recherche est divisé par 2).

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765